Code Analysis – Challenge Solution: Wonderland 4

By: Tehila Chananayev

Introduction

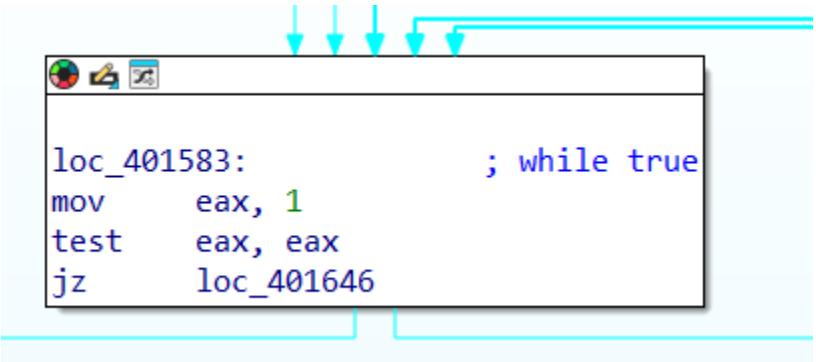
This challenge presents a piece of assembly code where the user enters a number, and the program checks whether the input is correct. Our goal is to reverse engineer the logic, identify the test conditions, and find the correct number that passes verification with the exercise requirement (no cracking in any way. No code changes, no registers, no memory...)

Code Structure

The beginning of the code begins a while (true) loop using:

---

test 1, 1



Since test 1, 1 always returns a non-zero result (sets ZF = 0), the condition is always true, creating an infinite loop — a classic way of implementing while (true) in Assembly.

After receiving the following code, it receives a numeric value from the user using the scenf function and also using %du. This can be seen

```
lea     ecx, [ebp+user_input]
push    ecx             ; char
push    offset aDu      ; " %du"
call    scanf
add     esp, 8
cmp     eax, 1
jz      short loc_4015B5
```

Core Logic

The following code compares two values using the strncmp function. The value stored in the ecx register contains the memory address ebp + str, which is 0019FEF8, and it holds the string "Good luck!!". On the other hand, the edx register contains the user input.

The comparison checks if the string located at ecx matches the input stored in edx. If the comparison is successful, the program proceeds as expected.

```
loc_4015B5:
mov     edx, 4
shl     edx, 0
mov     dword ptr [ebp+edx+Str], 63756C20h
mov     eax, 4
shl     eax, 1
mov     dword ptr [ebp+eax+Str], 21216Bh
mov     ecx, 4
imul    edx, ecx, 0
mov     dword ptr [ebp+edx+Str], 646F6F47h
lea     eax, [ebp+Str]
push    eax             ; Str
call    strlen
add     esp, 4
push    eax             ; MaxCount
lea     ecx, [ebp+Str]
push    ecx             ; Str2
mov     edx, [ebp+user_input]
push    edx             ; Str1
call    ds:strncmp
add     esp, 0Ch
test    eax, eax
jz      short check
```

I tried to input the address where the string is stored, specifically EBP + STR, to compare the exact same string from the same memory location. The initial check passed successfully. However, there was another check that caused an issue.

```
check:
lea     eax, [ebp+Str]
cmp     [ebp+user_input], eax
jnz     short loc_401632
```

This additional check verifies that the strings are not located at the same memory address. As a result, I had to search for the same string elsewhere in memory. Eventually, I found it at a different address, which allowed the check to pass.

Therefore, I went to memory using Alt + B, where I entered the string "Good luck!!". I discovered that the string appeared within another string, "Yeah! Good luck!! (and good job!)\r\n", which contained the substring I needed. I then extracted just the "Good luck!!" part, which was located at the address 40473E in hexadecimal, and **4212542** in decimal. This allowed me to pass the check, as it confirmed that the same string existed at different memory locations.

```
00404730  77 61 79 2E 0D 0A 00 00  59 65 61 68 21 20 47 6F  way.....Yeah!·Go
00404740  6F 64 20 6C 75 63 6B 21  21 20 28 61 6E 64 20 67  od·luck!!·(and·g
00404750  6F 6F 64 20 6A 6F 62 21  29 0D 0A 00 57 72 6F 6E  ood·job!)...Wron
00404760  67 21 0D 0A 00 00 00 00  59 6F 75 20 6D 61 79 20  g!......You·may·
00404770  65 6E 74 65 72 2C 20 62  75 74 20 63 61 6E 20 79  enter,·but·can·y
```

```
Welcome to Wonderland. I am the mad hatter, and I have some riddles for you...
Input a level number (latest level- 4):
4


Wait... I have something on the tip of my tongue!
(Enter the correct number)
4212542
Yeah! Good luck!! (and good job!)
```